

38 LinkedList Find the middle Element of a linked list.

```
class Solution {
public:
    ListNode* middleNode(ListNode* head) {
        ListNode* slow = head;
        ListNode* fast = head;

        while (fast && fast->next) {
            slow = slow->next;
            fast = fast->next->next;
        }

        return slow; // middle node
    }
};
```

The screenshot displays a LeetCode submission interface. On the left, the 'Accepted' status is shown with 36/36 testcases passed. The runtime is 0 ms, beating 100.00% of submissions, and the memory usage is 10.05 MB, beating 25.25%. A bar chart shows the runtime performance relative to other submissions. The right side of the page shows the C++ code for the solution, which uses the two-pointer technique to find the middle node of a linked list. The test results section shows that the solution passed all test cases, including Case 1 and Case 2.

Accepted 36 / 36 testcases passed
slockahuja submitted at Nov 18, 2025 02:30

Runtime 0 ms | Beats 100.00%
Memory 10.05 MB | Beats 25.25%

Code | C++

```
class Solution {
public:
    ListNode* middleNode(ListNode* head) {
        ListNode* slow = head;
        ListNode* fast = head;

        while (fast && fast->next) {
            slow = slow->next;
            fast = fast->next->next;
        }

        return slow; // middle node
    }
};
```

Testcase **Test Result**

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

head =
[1,2,3,4,5]

Output

[3,4,5]

Expected